

Lab 02: Computer Vision Topics - Preparation Guide

Medical Imaging Portfolio: Chest X-Ray Enhancement, CT/MRI Fusion, Echocardiogram Video Analysis

Lab 2 is **point-wise intensity processing**: every operation changes a pixel's value using only that pixel or the image's global statistics. There are no neighbourhoods or filters. The pipeline is:

load -> equalize -> colormap -> color balance -> threshold -> log -> gamma -> fuse -> process video frame

0. Foundation: what an image is

Concept. A grayscale image is a 2D NumPy array of uint8 values from 0 (black) to 255 (white). A color image is a 3D array (H, W, 3). **OpenCV stores channels as BGR, not RGB.**

When it matters. Every operation here is arithmetic on that array. BGR vs RGB is the most common bug, because Matplotlib expects RGB and OpenCV gives BGR.

```
raw = cv2.imread('data/sample_xray.png', cv2.IMREAD_GRAYSCALE) # 2D array
print(raw.shape, raw.dtype, raw.min(), raw.max())
plt.imshow(cv2.cvtColor(bgr_img, cv2.COLOR_BGR2RGB)) # fix channels before showing
```

- Use **relative paths** (data/...), as the submission guidelines require.
- imread returns None on failure, so assert raw is not None.

1. Histogram and histogram equalization (Tasks 1, 2, 3)

Concept. A histogram counts how many pixels have each intensity (0-255). An underexposed X-ray has all its pixels crowded into a narrow dark band, which means low contrast. **Histogram equalization** spreads that band across the full 0-255 range using the cumulative distribution function (CDF) of the histogram as the mapping: $\text{new_value} = \text{CDF}(\text{old_value}) \times 255$. Common intensities get stretched apart, so contrast increases where most data lives.

When to use it

- The image is low-contrast, underexposed or murky (X-rays, ultrasound).
- You want to normalize dynamic range before blending or colormapping, as in Task 2 where CT and MRI are equalized separately before fusion.

Limitations (viva favourite)

- It is automatic and global, with no parameter to tune.
- It can amplify noise, which matters for ultrasound.
- It only works on single-channel images. For color, equalize the luminance channel instead.

How to use it

```
equalized = cv2.equalizeHist(raw) # input must be 8-bit single-channel
plt.hist(raw.ravel(), bins=256, range=(0,255)) # plot before vs after to prove it worked
```

Observation to write: The raw histogram was crowded in a narrow dark band. After equalization it spreads across the full range, and rib and lung boundaries became visible.

2. Color mapping / false color (Tasks 1, 2, 3)

Concept. The eye distinguishes roughly 30 shades of gray but thousands of colors. A colormap (lookup table) assigns each gray value 0-255 a color. COLORMAP_JET runs blue (low), cyan, green, yellow, red (high). This is **pseudo-color or false-color** imaging. No new information is added; existing intensity differences just become perceptible.

When to use it

- To spot subtle intensity boundaries such as fluid edges, or blood-flow intensity in ultrasound.
- Not to claim that red means a specific disease. The colors are arbitrary.

```
heatmap = cv2.applyColorMap(equalized, cv2.COLORMAP_JET) # input: 8-bit gray, output: BGR
```

- Task 2 uses COLORMAP_BONE for CT (stays close to gray, so edges stay legible) and JET for MRI (makes soft-tissue variation pop).
- The output is now **3-channel**, so everything after this step operates on color.

3. Color balance (Tasks 1, 3)

Concept. A color cast is when one channel dominates, giving a tint such as too much blue. **Gray-world assumption:** in a natural image the average of R, G and B should be about equal. So scale each channel by `gray_mean / channel_mean` to neutralize the tint.

When to use it. After a colormap or lighting change that introduces a tint. This is the lab's 'simulate lighting correction' step.

```
b, g, r = cv2.split(img.astype(np.float64)) # float to avoid uint8 overflow
gray_mean = (b.mean()+g.mean()+r.mean())/3
b = np.clip(b * gray_mean/b.mean(), 0, 255) # same for g, r
cv2.merge([b,g,r]).astype(np.uint8)
```

Convert to float *before* multiplying and clip afterwards. A naive uint8 multiply wraps around (e.g. $200 \times 2 = 144$), which is a classic bug.

4. Thresholding / color filtering (Task 1)

Concept. Thresholding is the simplest segmentation. If `pixel > T` set it to 255, otherwise 0. The result is a binary **mask**. Dense tissue (bone, dense fluid) appears bright in X-rays, so a high threshold keeps only that tissue. You then **apply the mask** so only the selected pixels survive.

When to use it. To isolate a region by intensity. Choosing T: a fixed number (e.g. 200), a percentile (the 90th percentile adapts to each image), or Otsu's method (automatic).

```
T = np.percentile(equalized, 90) # keep the brightest 10%
_, mask = cv2.threshold(equalized, T, 255, cv2.THRESH_BINARY)
only_dense = cv2.bitwise_and(heatmap, heatmap, mask=mask) # mask applied to the image
```

`bitwise_and(img, img, mask=...)` is how you apply a mask. Pixels where the mask is 0 become black.

5. Logarithmic transformation (Tasks 1, 2, 3)

Concept. Formula: $s = c * \log(1 + r)$, with $c = 255 / \log(1 + \max)$.

- It maps a narrow range of dark inputs onto a wide range of outputs. Dark pixels get strongly brightened; bright pixels are compressed.

- The +1 avoids log(0).
- c scales the result back to 0-255.

When to use it. To reveal detail in the **dark regions**: faint rib edges (Task 1), dark heart chambers (Task 3), or an image whose data is crushed near black.

```
def log_transform(img):
    f = img.astype(np.float64)
    c = 255.0 / np.log(1 + f.max())
    return np.clip(c*np.log(1+f), 0, 255).astype(np.uint8)
```

Guard against $f.\max() == 0$ in video, since an all-black frame would divide by zero. The Task 3 code does this.

6. Power-law (gamma) transformation (Tasks 1, 2, 3)

Concept. Formula: $s = 255 * (r/255)^\gamma$. Normalize to 0-1, raise to a power, scale back.

gamma	Effect
< 1 (e.g. 0.6)	Brightens midtones and expands darks, like a milder log
= 1	No change
> 1	Darkens the image and suppresses brights

Unlike log, gamma has a **tunable parameter**, so it gives finer control. It is also how monitors historically work, hence the name 'gamma correction'.

When to use it

- $\gamma < 1$ to fine-tune midtones and reduce harsh bone contrast (Task 1).
- Task 3's brief says 'suppress blinding white backscatter'. Strictly, $\gamma > 1$ darkens brights, while $\gamma = 0.7$ brightens midtones. If asked, explain what you chose and why.

```
def gamma_transform(img, gamma=0.6):
    return np.clip(255*np.power(img/255.0, gamma), 0, 255).astype(np.uint8)
```

Alternative: precompute a 256-entry lookup table and call `cv2.LUT()`, which is faster for video.

Log vs gamma (likely question). Both are non-linear point operations. Log is a fixed curve that strongly boosts darks. Gamma is a family of curves controlled by gamma, so it is more flexible.

7. Image fusion with weighted blending (Task 2)

Concept. Different modalities capture different information. **CT** shows sharp structure and bone edges. **MRI** shows soft-tissue detail. Fusion combines them using **linear blending**: $\text{fused} = \alpha * A + \beta * B + \gamma$. Prerequisite: the images must be **registered**, meaning pixel-aligned and the same shape (the code asserts `ct.shape == mri.shape`).

When to use it. Combining complementary sources of the same scene (multi-modal, multi-exposure, overlays). Weights show priority: CT $\alpha = 0.7$ preserves edges, MRI $\beta = 0.3$ adds tissue variation.

```
fused = cv2.addWeighted(ct_color, 0.7, mri_color, 0.3, 0)
```

- Both inputs must have the **same size and dtype/channels**.
- `addWeighted` **saturates**: values above 255 clip rather than wrap.
- Weights summing to 1 keep brightness unchanged. A sum above 1 gives washed-out whites.

- That is why the brief asks for log and gamma afterwards: to recover any crushed or blown-out detail.

Comparative analysis. Show CT | MRI | Fused side by side with `plt.subplots(1,3)` and state that the edges came from CT and the tissue contrast from MRI.

8. Video processing (Task 3)

Concept. A video is a sequence of frames, so you apply the same image pipeline to each one. 'Real-time' means each frame is processed fast enough to display as it arrives.

```
cap = cv2.VideoCapture('data/echo_sample.mp4') # file path, not 0 (webcam)
assert cap.isOpened()
while True:
    ret, frame = cap.read() # ret=False -> end of video
    if not ret: break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # equalizeHist needs 1 channel
    enhanced = enhance_frame(gray)
    combined = np.hstack([raw_bgr, enhanced]) # side by side
    cv2.imshow('Raw | Enhanced', combined)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

Things to be able to explain

- **np.hstack** needs equal height and equal channel count. That is why the gray raw frame is converted back to BGR with `COLOR_GRAY2BGR` before stacking against the 3-channel enhanced frame.
- **cv2.waitKey(1)** gives the window time to refresh and polls the keyboard. Without it, nothing shows.
- **cap.release()** frees the file handle.
- `cap.get(cv2.CAP_PROP_FPS)` and `cv2.CAP_PROP_FRAME_COUNT` give video metadata. The clip is 215 frames at 50 fps.
- **Pipeline order matters.** The order is equalize, colormap, balance, log, gamma, and the first step needs grayscale input.
- Saving screenshots with `cv2.imwrite` is a fallback when `imshow` cannot run headlessly.

Quick reference: when to use which

Problem	Tool	Function
Low contrast, narrow histogram	Histogram equalization	<code>cv2.equalizeHist</code>
Subtle gray differences hard to see	Color mapping	<code>cv2.applyColorMap</code>
Tinted or color-cast image	Gray-world balance	channel scaling by hand
Isolate bright/dense regions	Threshold + mask	<code>cv2.threshold</code> , <code>cv2.bitwise_and</code>
Details hidden in dark areas	Log transform	<code>c*np.log(1+r)</code>
Fine control of midtones/brights	Gamma	<code>255*(r/255)**gamma</code>
Combine two aligned images	Weighted fusion	<code>cv2.addWeighted</code>
Process a video	Capture loop	<code>VideoCapture</code> , <code>read()</code> , <code>imshow</code> , <code>waitKey</code>

Common pitfalls

- **BGR vs RGB.** Convert before `plt.imshow`.
- **uint8 overflow.** Convert to float for arithmetic, then clip and cast back.
- **equalizeHist needs grayscale 8-bit.** It fails on color images.
- **addWeighted needs matching shapes** and channel counts.
- **Order of operations.** Equalize before colormapping, because the colormap output is 3-channel.
- **Absolute paths** break the submission. Use `data/...`
- **Forgetting `cap.release()`** and `destroyAllWindows()`.

Likely viva questions

1. Why does equalization help an underexposed image, and what is its downside? (It spreads the narrow histogram using the CDF. It can amplify noise.)
2. Why can't you equalize each color channel separately for a color image? (It shifts the hues.)
3. Is false color adding information? (No. It only improves perceptibility.)
4. What does $\gamma < 1$ do versus $\gamma > 1$?
5. Why $\log(1+r)$ and not $\log(r)$? ($\log(0)$ is undefined.)
6. Why equalize CT and MRI independently before fusing?
7. Why must the images be the same size to use `addWeighted`?
8. What does `waitKey(1)` do?
9. What is the gray-world assumption?